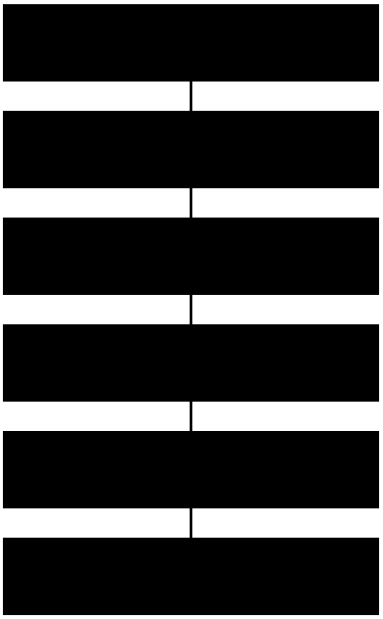
PrepHQ Online Judge System

Complete Technical Architecture & Execution Flow

This document explains the complete Docker-based Online Judge architecture including database design, request lifecycle, harness runner, judge system, Docker execution, security controls and final verdict generation.

High Level Architecture



User code travels through the controller, harness runner and Docker executor before being executed inside an isolated runtime container.

Database Design

QUESTIONS TABLE	Purpose
id	Primary Key
title	Question title
difficulty	Easy/Medium/Hard
description	Problem statement
starter_code	Language templates
entry_point	Function name to call
param_order	Function parameters

TESTCASES TABLE	Purpose
question_id	Question mapping
inputs	Function inputs
expected_output	Expected result
is_hidden	Hidden testcase flag

Questions define metadata while testcases define validation logic used by the judge.

Execution Lifecycle

1. User submits code.
2. Run Controller validates payload.
3. Harness Runner detects language.
4. Temporary workspace is created.
5. buildJSHarness()/buildPyHarness() generates wrapper code.
6. Main.js/Main.py is written into workspace.
7. runDockerCommand() creates docker run command.
8. Docker Engine starts isolated container.
9. Testcases are executed.
10. __JUDGE__ result is produced.
11. Outputs are compared with expected outputs.
12. Accepted / Wrong Answer / Runtime Error is returned.
13. Workspace cleanup occurs.

Harness Runner Deep Dive

buildJSHarness() wraps user code and injects predefined testcases automatically. The user only writes the function. The harness calls that function, measures runtime and collects results.

buildPyHarness() performs the same task for Python submissions.

normalizedEqual() compares actual and expected outputs safely.

__JUDGE__ Marker is a custom marker used to identify structured evaluation results among normal console output.

Docker Security Model

Flag	Purpose
--network none	Disable internet access
--memory 256m	Limit memory usage
--cpus 1	Limit CPU usage
--read-only	Protect filesystem
--pids-limit 64	Prevent process abuse
--rm	Auto remove container

Without Docker, user code would execute directly on the server and could impact system stability. Docker provides isolation, security and resource control.

Manager Summary

PrepHQ uses a Docker-based Online Judge architecture. Run Controller validates requests, Harness Runner injects predefined testcases, Docker Executor launches isolated containers, Judge Logic evaluates outputs and returns verdicts. The system supports multiple languages, resource limits and secure execution.